

6. Built-in Modules

Python takes a “batteries included” approach to the standard library. Many other languages ship with a small number of common packages and require you to look elsewhere for important functionality. Although Python also has an impressive repository of community-built modules, it strives to provide, in its default installation, the most important modules for common uses of the language.

The full set of standard modules is too large to cover in this book. But some of these built-in packages are so closely intertwined with idiomatic Python that they may as well be part of the language specification. These essential built-in modules are especially important when writing the intricate, error-prone parts of programs.

Item 42: Define Function Decorators with `functools.wraps`

Python has special syntax for *decorators* that can be applied to functions. Decorators have the ability to run additional code before and after any calls to the functions they wrap. This allows them to access and modify input arguments and return values. This functionality can be useful for enforcing semantics, debugging, registering functions, and more.

For example, say you want to print the arguments and return value of a function call. This is especially helpful when debugging a stack of function calls from a recursive function. Here, I define such a decorator:

[Click here to view code image](#)

```
def trace(func):
    def wrapper(*args, **kwargs):
        result = func(*args, **kwargs)
        print('%s(%r, %r) -> %r' %
              (func.__name__, args, kwargs, result))
        return result
    return wrapper
```

I can apply this to a function using the @ symbol.

[Click here to view code image](#)

```
@trace
def fibonacci(n):
    """Return the n-th Fibonacci number"""
    if n in (0, 1):
        return n
    return (fibonacci(n - 2) + fibonacci(n - 1))
```

The @ symbol is equivalent to calling the decorator on the function it wraps and assigning the return value to the original name in the same scope.

```
fibonacci = trace(fibonacci)
```

Calling this decorated function will run the `wrapper` code before and after `fibonacci` runs, printing the arguments and return value at each level in the recursive stack.

```
fibonacci(3)
```

```
>>>
fibonacci((1,), {}) -> 1
fibonacci((0,), {}) -> 0
fibonacci((1,), {}) -> 1
fibonacci((2,), {}) -> 1
fibonacci((3,), {}) -> 2
```

This works well, but it has an unintended side effect. The value returned by the decorator—the function that’s called above—doesn’t think it’s named `fibonacci`.

[Click here to view code image](#)

```
print(fibonacci)
>>>
<function trace.<locals>.wrapper at 0x107f7ed08>
```

The cause of this isn’t hard to see. The `trace` function returns the `wrapper` it defines. The `wrapper` function is what’s assigned to the `fibonacci` name in the containing module because of the decorator. This behavior is problematic because it undermines tools that do introspection, such as debuggers (see [Item 57: “Consider Interactive Debugging with `pdb`”](#)) and object serializers (see [Item 44: “Make `pickle` Reliable with `copyreg`”](#)).

For example, the `help` built-in function is useless on the decorated `fibonacci` function.

[Click here to view code image](#)

```
help(fibonacci)
>>>
Help on function wrapper in module __main__:

wrapper(*args, **kwargs)
```

The solution is to use the `wraps` helper function from the `functools` built-in module. This is a decorator that helps you write decorators. Applying it to the `wrapper` function will copy all of the important metadata about the inner function to the outer function.

[Click here to view code image](#)

```
def trace(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        # ...
    return wrapper

@trace
def fibonacci(n):
    # ...
```

Now, running the `help` function produces the expected result, even though the function is decorated.

[Click here to view code image](#)

```
help(fibonacci)
>>>
Help on function fibonacci in module __main__:

fibonacci(n)
```

Return the n-th Fibonacci number

Calling `help` is just one example of how decorators can subtly cause problems. Python functions have many other standard attributes (e.g., `__name__`, `__module__`) that must be preserved to maintain the interface of functions in the language. Using `wraps` ensures that you'll always get the correct behavior.

Things to Remember

- ◆ Decorators are Python syntax for allowing one function to modify another function at runtime.
- ◆ Using decorators can cause strange behaviors in tools that do introspection, such as debuggers.
- ◆ Use the `wraps` decorator from the `functools` built-in module when you define your own decorators to avoid any issues.

Item 43: Consider `contextlib` and `with` Statements for Reusable `try/finally` Behavior

The `with` statement in Python is used to indicate when code is running in a special context. For example, mutual exclusion locks (see [Item 38: “Use `Lock` to Prevent Data Races in Threads](#)”) can be used in `with` statements to indicate that the indented code only runs while the lock is held.

```
lock = Lock()
with lock:
    print('Lock is held')
```

The example above is equivalent to this `try/finally` construction because the `Lock` class properly enables the `with` statement.

```
lock.acquire()
try:
    print('Lock is held')
finally:
    lock.release()
```

The `with` statement version of this is better because it eliminates the need to write the repetitive code of the `try/finally` construction. It's easy to make your objects and functions capable of use in `with` statements by using the `contextlib` built-in module. This module contains the `contextmanager` decorator, which lets a simple function be used in `with` statements. This is much easier than defining a new class with the special methods `__enter__` and `__exit__` (the standard way).

For example, say you want a region of your code to have more debug logging sometimes. Here, I define a function that does logging at two severity levels:

[Click here to view code image](#)

```
def my_function():
    logging.debug('Some debug data')
    logging.error('Error log here')
```

```
logging.debug('More debug data')
```

The default log level for my program is **WARNING**, so only the error message will print to screen when I run the function.

```
my_function()
>>>
Error log here
```

I can elevate the log level of this function temporarily by defining a context manager. This helper function boosts the logging severity level before running the code in the `with` block and reduces the logging severity level afterward.

[Click here to view code image](#)

```
@contextmanager
def debug_logging(level):
    logger = logging.getLogger()
    old_level = logger.getEffectiveLevel()
    logger.setLevel(level)
    try:
        yield
    finally:
        logger.setLevel(old_level)
```

The `yield` expression is the point at which the `with` block's contents will execute. Any exceptions that happen in the `with` block will be re-raised by the `yield` expression for you to catch in the helper function (see [Item 40: “Consider Coroutines to Run Many Functions Concurrently”](#) for an explanation of how that works).

Now, I can call the same logging function again, but in the `debug_logging` context. This time, all of the debug messages are printed to the screen during the `with` block. The same function running outside the `with` block won't print debug messages.

[Click here to view code image](#)

```
with debug_logging(logging.DEBUG):
    print('Inside:')
    my_function()
print('After:')
my_function()

>>>
Inside:
Some debug data
Error log here
More debug data
After:
Error log here
```

Using `with` Targets

The context manager passed to a `with` statement may also return an object. This object is assigned to a local variable in the `as` part of the compound statement. This gives the code running in the `with` block the ability to directly interact with its context.

For example, say you want to write a file and ensure that it's always closed correctly. You can do this by passing `open` to the `with` statement. `open` returns a file handle for the `as`

target of `with` and will close the handle when the `with` block exits.

[Click here to view code image](#)

```
with open('/tmp/my_output.txt', 'w') as handle:
    handle.write('This is some data!')
```

This approach is preferable to manually opening and closing the file handle every time. It gives you confidence that the file is eventually closed when execution leaves the `with` statement. It also encourages you to reduce the amount of code that executes while the file handle is open, which is good practice in general.

To enable your own functions to supply values for `as` targets, all you need to do is `yield` a value from your context manager. For example, here I define a context manager to fetch a `Logger` instance, set its level, and then `yield` it for the `as` target.

[Click here to view code image](#)

```
@contextmanager
def log_level(level, name):
    logger = logging.getLogger(name)
    old_level = logger.getEffectiveLevel()
    logger.setLevel(level)
    try:
        yield logger
    finally:
        logger.setLevel(old_level)
```

Calling logging methods like `debug` on the `as` target will produce output because the logging severity level is set low enough in the `with` block. Using the `logging` module directly won't print anything because the default logging severity level for the default program logger is `WARNING`.

[Click here to view code image](#)

```
with log_level(logging.DEBUG, 'my-log') as logger:
    logger.debug('This is my message!')
    logging.debug('This will not print')
```

```
>>>
This is my message!
```

After the `with` statement exits, calling debug logging methods on the `Logger` named `'my-log'` will not print anything because the default logging severity level has been restored. Error log messages will always print.

[Click here to view code image](#)

```
logger = logging.getLogger('my-log')
logger.debug('Debug will not print')
logger.error('Error will print')
```

```
>>>
Error will print
```

Things to Remember

- ◆ The `with` statement allows you to reuse logic from `try/finally` blocks and reduce visual noise.

- ◆ The `contextlib` built-in module provides a `contextmanager` decorator that makes it easy to use your own functions in `with` statements.
- ◆ The value yielded by context managers is supplied to the `as` part of the `with` statement. It's useful for letting your code directly access the cause of the special context.

Item 44: Make `pickle` Reliable with `copyreg`

The `pickle` built-in module can serialize Python objects into a stream of bytes and deserialize bytes back into objects. Pickled byte streams shouldn't be used to communicate between untrusted parties. The purpose of `pickle` is to let you pass Python objects between programs that you control over binary channels.

Note

The `pickle` module's serialization format is unsafe by design. The serialized data contains what is essentially a program that describes how to reconstruct the original Python object. This means a malicious `pickle` payload could be used to compromise any part of the Python program that attempts to deserialize it.

In contrast, the `json` module is safe by design. Serialized JSON data contains a simple description of an object hierarchy. Deserializing JSON data does not expose a Python program to any additional risk. Formats like JSON should be used for communication between programs or people that don't trust each other.

For example, say you want to use a Python object to represent the state of a player's progress in a game. The game state includes the level the player is on and the number of lives he or she has remaining.

```
class GameState(object):
    def __init__(self):
        self.level = 0
        self.lives = 4
```

The program modifies this object as the game runs.

[Click here to view code image](#)

```
state = GameState()
state.level += 1 # Player beat a level
state.lives -= 1 # Player had to try again
```

When the user quits playing, the program can save the state of the game to a file so it can be resumed at a later time. The `pickle` module makes it easy to do this. Here, I dump the `GameState` object directly to a file:

[Click here to view code image](#)

```
state_path = '/tmp/game_state.bin'
with open(state_path, 'wb') as f:
    pickle.dump(state, f)
```

Later, I can load the file and get back the `GameState` object as if it had never been serialized.

[Click here to view code image](#)

```
with open(state_path, 'rb') as f:
    state_after = pickle.load(f)
print(state_after.__dict__)

>>>
{'lives': 3, 'level': 1}
```

The problem with this approach is what happens as the game's features expand over time. Imagine you want the player to earn points towards a high score. To track the player's points, you'd add a new field to the `GameState` class.

```
class GameState(object):
    def __init__(self):
        # ...
        self.points = 0
```

Serializing the new version of the `GameState` class using `pickle` will work exactly as before. Here, I simulate the round-trip through a file by serializing to a string with `dumps` and back to an object with `loads`:

[Click here to view code image](#)

```
state = GameState()
serialized = pickle.dumps(state)
state_after = pickle.loads(serialized)
print(state_after.__dict__)

>>>
{'lives': 4, 'level': 0, 'points': 0}
```

But what happens to older saved `GameState` objects that the user may want to resume? Here, I unpickle an old game file using a program with the new definition of the `GameState` class:

[Click here to view code image](#)

```
with open(state_path, 'rb') as f:
    state_after = pickle.load(f)
print(state_after.__dict__)

>>>
{'lives': 3, 'level': 1}
```

The `points` attribute is missing! This is especially confusing because the returned object is an instance of the new `GameState` class.

[Click here to view code image](#)

```
assert isinstance(state_after, GameState)
```

This behavior is a byproduct of the way the `pickle` module works. Its primary use case is making it easy to serialize objects. As soon as your use of `pickle` expands beyond trivial usage, the module's functionality starts to break down in surprising ways.

Fixing these problems is straightforward using the `copyreg` built-in module. The `copyreg` module lets you register the functions responsible for serializing Python objects, allowing you to control the behavior of `pickle` and make it more reliable.

Default Attribute Values

In the simplest case, you can use a constructor with default arguments (see [Item 19: “Provide Optional Behavior with Keyword Arguments”](#)) to ensure that `GameState` objects will always have all attributes after unpickling. Here, I redefine the constructor this way:

[Click here to view code image](#)

```
class GameState(object):
    def __init__(self, level=0, lives=4, points=0):
        self.level = level
        self.lives = lives
        self.points = points
```

To use this constructor for pickling, I define a helper function that takes a `GameState` object and turns it into a tuple of parameters for the `copyreg` module. The returned tuple contains the function to use for unpickling and the parameters to pass to the unpickling function.

[Click here to view code image](#)

```
def pickle_game_state(game_state):
    kwargs = game_state.__dict__
    return unpickle_game_state, (kwargs,)
```

Now, I need to define the `unpickle_game_state` helper. This function takes serialized data and parameters from `pickle_game_state` and returns the corresponding `GameState` object. It's a tiny wrapper around the constructor.

[Click here to view code image](#)

```
def unpickle_game_state(kwargs):
    return GameState(**kwargs)
```

Now, I register these with the `copyreg` built-in module.

[Click here to view code image](#)

```
copyreg.pickle(GameState, pickle_game_state)
```

Serializing and deserializing works as before.

[Click here to view code image](#)

```
state = GameState()
state.points += 1000
serialized = pickle.dumps(state)
state_after = pickle.loads(serialized)
print(state_after.__dict__)
```

```
>>>
{'lives': 4, 'level': 0, 'points': 1000}
```

With this registration done, now I can change the definition of `GameState` to give the player a count of magic spells to use. This change is similar to when I added the `points` field to `GameState`.

[Click here to view code image](#)

```
class GameState(object):
    def __init__(self, level=0, lives=4, points=0, magic=5):
```



```
# ...
```

But unlike before, deserializing an old `GameState` object will result in valid game data instead of missing attributes. This works because `unpickle_game_state` calls the `GameState` constructor directly. The constructor's keyword arguments have default values when parameters are missing. This causes old game state files to receive the default value for the new `magic` field when they are deserialized.

[Click here to view code image](#)

```
state_after = pickle.loads(serialized)
print(state_after.__dict__)

>>>
{'level': 0, 'points': 1000, 'magic': 5, 'lives': 4}
```

Versioning Classes

Sometimes you'll need to make backwards-incompatible changes to your Python objects by removing fields. This prevents the default argument approach to serialization from working.

For example, say you realize that a limited number of lives is a bad idea, and you want to remove the concept of lives from the game. Here, I redefine the `GameState` to no longer have a `lives` field:

[Click here to view code image](#)

```
class GameState(object):
    def __init__(self, level=0, points=0, magic=5):
        # ...
```

The problem is that this breaks deserializing old game data. All fields from the old data, even ones removed from the class, will be passed to the `GameState` constructor by the `unpickle_game_state` function.

[Click here to view code image](#)

```
pickle.loads(serialized)

>>>
TypeError: __init__() got an unexpected keyword argument 'lives'
```

The solution is to add a `version` parameter to the functions supplied to `copyreg`. New serialized data will have a version of 2 specified when pickling a new `GameState` object.

[Click here to view code image](#)

```
def unpickle_game_state(game_state):
    kwargs = game_state.__dict__
    kwargs['version'] = 2
    return GameState(**kwargs)
```

Old versions of the data will not have a `version` argument present, allowing you to manipulate the arguments passed to the `GameState` constructor accordingly.

[Click here to view code image](#)

```
def unpickle_game_state(kwargs):
```

```

version = kwargs.pop('version', 1)
if version == 1:
    kwargs.pop('lives')
return GameState(**kwargs)

```

Now, deserializing an old object works properly.

[Click here to view code image](#)

```

copyreg.pickle(GameState, pickle_game_state)
state_after = pickle.loads(serialized)
print(state_after.__dict__)

>>>
{'magic': 5, 'level': 0, 'points': 1000}

```

You can continue this approach to handle changes between future versions of the same class. Any logic you need to adapt an old version of the class to a new version of the class can go in the `unpickle_game_state` function.

Stable Import Paths

One other issue you may encounter with `pickle` is breakage from renaming a class. Often over the life cycle of a program, you'll refactor your code by renaming classes and moving them to other modules. Unfortunately, this will break the `pickle` module unless you're careful.

Here, I rename the `GameState` class to `BetterGameState`, removing the old class from the program entirely:

[Click here to view code image](#)

```

class BetterGameState(object):
    def __init__(self, level=0, points=0, magic=5):
        # ...

```

Attempting to deserialize an old `GameState` object will now fail because the class can't be found.

[Click here to view code image](#)

```

pickle.loads(serialized)
>>>
AttributeError: Can't get attribute 'GameState' on <module '__main__' from
'my_code.py'>

```

The cause of this exception is that the import path of the serialized object's class is encoded in the pickled data.

[Click here to view code image](#)

```

print(serialized[:25])
>>>
b'\x80\x03c__main__\nGameState\nq\x00)'

```

The solution is to use `copyreg` again. You can specify a stable identifier for the function to use for unpickling an object. This allows you to transition pickled data to different classes with different names when it's deserialized. It gives you a level of indirection.

[Click here to view code image](#)

```
copyreg.pickle(BetterGameState, pickle_game_state)
```

After using `copyreg`, you can see that the import path to `pickle_game_state` is encoded in the serialized data instead of `BetterGameState`.

[Click here to view code image](#)

```
state = BetterGameState()
serialized = pickle.dumps(state)
print(serialized[:35])

>>>
b'\x80\x03c__main__\nunpickle_game_state\nq\x00{'
```

The only gotcha is that you can't change the path of the module in which the `unpickle_game_state` function is present. Once you serialize data with a function, it must remain available on that import path for deserializing in the future.

Things to Remember

- ◆ The `pickle` built-in module is only useful for serializing and deserializing objects between trusted programs.
- ◆ The `pickle` module may break down when used for more than trivial use cases.
- ◆ Use the `copyreg` built-in module with `pickle` to add missing attribute values, allow versioning of classes, and provide stable import paths.

Item 45: Use `datetime` Instead of `time` for Local Clocks

Coordinated Universal Time (UTC) is the standard, time-zone-independent representation of time. UTC works great for computers that represent time as seconds since the UNIX epoch. But UTC isn't ideal for humans. Humans reference time relative to where they're currently located. People say "noon" or "8 am" instead of "UTC 15:00 minus 7 hours." If your program handles time, you'll probably find yourself converting time between UTC and local clocks to make it easier for humans to understand.

Python provides two ways of accomplishing time zone conversions. The old way, using the `time` built-in module, is disastrously error prone. The new way, using the `datetime` built-in module, works great with some help from the community-built package named `pytz`.

You should be acquainted with both `time` and `datetime` to thoroughly understand why `datetime` is the best choice and `time` should be avoided.

The `time` Module

The `localtime` function from the `time` built-in module lets you convert a UNIX timestamp (seconds since the UNIX epoch in UTC) to a local time that matches the host computer's time zone (Pacific Daylight Time, in my case).

[Click here to view code image](#)

```
from time import localtime, strftime
now = 1407694710
```

```
local_tuple = localtime(now)
time_format = '%Y-%m-%d %H:%M:%S'
time_str = strftime(time_format, local_tuple)
print(time_str)
```

```
>>>
```

```
2014-08-10 11:18:30
```

You'll often need to go the other way as well, starting with user input in local time and converting it to UTC time. You can do this by using the `strptime` function to parse the time string, then call `mktime` to convert local time to a UNIX timestamp.

[Click here to view code image](#)

```
from time import mktime, strptime

time_tuple = strptime(time_str, time_format)
utc_now = mktime(time_tuple)
print(utc_now)
```

```
>>>
```

```
1407694710.0
```

How do you convert local time in one time zone to local time in another? For example, say you are taking a flight between San Francisco and New York, and want to know what time it will be in San Francisco once you've arrived in New York.

Directly manipulating the return values from the `time`, `localtime`, and `strptime` functions to do time zone conversions is a bad idea. Time zones change all the time due to local laws. It's too complicated to manage yourself, especially if you want to handle every global city for flight departure and arrival.

Many operating systems have configuration files that keep up with the time zone changes automatically. Python lets you use these time zones through the `time` module. For example, here I parse the departure time from the San Francisco time zone of Pacific Daylight Time:

[Click here to view code image](#)

```
parse_format = '%Y-%m-%d %H:%M:%S %Z'
depart_sfo = '2014-05-01 15:45:16 PDT'
time_tuple = strptime(depart_sfo, parse_format)
time_str = strftime(time_format, time_tuple)
print(time_str)
```

```
>>>
```

```
2014-05-01 15:45:16
```

After seeing that PDT works with the `strptime` function, you might also assume that other time zones known to my computer will also work. Unfortunately, this isn't the case. Instead, `strptime` raises an exception when it sees Eastern Daylight Time (the time zone for New York).

[Click here to view code image](#)

```
arrival_nyc = '2014-05-01 23:33:24 EDT'
time_tuple = strptime(arrival_nyc, time_format)
```

```
>>>
```

ValueError: unconverted data remains: EDT

The problem here is the platform-dependent nature of the `time` module. Its actual behavior is determined by how the underlying C functions work with the host operating system. This makes the functionality of the `time` module unreliable in Python. The `time` module fails to consistently work properly for multiple local times. Thus, you should avoid the `time` module for this purpose. If you must use `time`, only use it to convert between UTC and the host computer's local time. For all other types of conversions, use the `datetime` module.

The datetime Module

The second option for representing times in Python is the `datetime` class from the `datetime` built-in module. Like the `time` module, `datetime` can be used to convert from the current time in UTC to local time.

Here, I take the present time in UTC and convert it to my computer's local time (Pacific Daylight Time):

[Click here to view code image](#)

```
from datetime import datetime, timezone

now = datetime(2014, 8, 10, 18, 18, 30)
now_utc = now.replace(tzinfo=timezone.utc)
now_local = now_utc.astimezone()
print(now_local)

>>>
2014-08-10 11:18:30-07:00
```

The `datetime` module can also easily convert a local time back to a UNIX timestamp in UTC.

[Click here to view code image](#)

```
time_str = '2014-08-10 11:18:30'
now = datetime.strptime(time_str, time_format)
time_tuple = now.timetuple()
utc_now = mktime(time_tuple)
print(utc_now)

>>>
1407694710.0
```

Unlike the `time` module, the `datetime` module has facilities for reliably converting from one local time to another local time. However, `datetime` only provides the machinery for time zone operations with its `tzinfo` class and related methods. What's missing are the time zone definitions besides UTC.

Luckily, the Python community has addressed this gap with the `pytz` module that's available for download from the Python Package Index (<https://pypi.python.org/pypi/pytz/>). `pytz` contains a full database of every time zone definition you might need.

To use `pytz` effectively, you should always convert local times to UTC first. Perform any

`datetime` operations you need on the UTC values (such as offsetting). Then, convert to local times as a final step.

For example, here I convert an NYC flight arrival time to a UTC `datetime`. Although some of these calls seem redundant, all of them are necessary when using `pytz`.

[Click here to view code image](#)

```
arrival_nyc = '2014-05-01 23:33:24'
nyc_dt_naive = datetime.strptime(arrival_nyc, time_format)
eastern = pytz.timezone('US/Eastern')
nyc_dt = eastern.localize(nyc_dt_naive)
utc_dt = pytz.utc.normalize(nyc_dt.astimezone(pytz.utc))
print(utc_dt)

>>>
2014-05-02 03:33:24+00:00
```

Once I have a UTC `datetime`, I can convert it to San Francisco local time.

[Click here to view code image](#)

```
pacific = pytz.timezone('US/Pacific')
sf_dt = pacific.normalize(utc_dt.astimezone(pacific))
print(sf_dt)

>>>
2014-05-01 20:33:24-07:00
```

Just as easily, I can convert it to the local time in Nepal.

[Click here to view code image](#)

```
nepal = pytz.timezone('Asia/Katmandu')
nepal_dt = nepal.normalize(utc_dt.astimezone(nepal))
print(nepal_dt)

>>>
2014-05-02 09:18:24+05:45
```

With `datetime` and `pytz`, these conversions are consistent across all environments regardless of what operating system the host computer is running.

Things to Remember

- ◆ Avoid using the `time` module for translating between different time zones.
- ◆ Use the `datetime` built-in module along with the `pytz` module to reliably convert between times in different time zones.
- ◆ Always represent time in UTC and do conversions to local time as the final step before presentation.

Item 46: Use Built-in Algorithms and Data Structures

When you're implementing Python programs that handle a non-trivial amount of data, you'll eventually see slowdowns caused by the algorithmic complexity of your code. This usually isn't the result of Python's speed as a language (see [Item 41: "Consider `concurrent.futures` for True Parallelism](#)" if it is). The issue, more likely, is that you aren't using the best algorithms and data structures for your problem.

Luckily, the Python standard library has many of the algorithms and data structures you'll need to use built in. Besides speed, using these common algorithms and data structures can make your life easier. Some of the most valuable tools you may want to use are tricky to implement correctly. Avoiding reimplementation of common functionality will save you time and headaches.

Double-ended Queue

The `deque` class from the `collections` module is a double-ended queue. It provides constant time operations for inserting or removing items from its beginning or end. This makes it ideal for first-in-first-out (FIFO) queues.

[Click here to view code image](#)

```
fifo = deque()
fifo.append(1)      # Producer
x = fifo.popleft()  # Consumer
```

The `list` built-in type also contains an ordered sequence of items like a queue. You can insert or remove items from the end of a list in constant time. But inserting or removing items from the head of a list takes linear time, which is much slower than the constant time of a `deque`.

Ordered Dictionary

Standard dictionaries are unordered. That means a `dict` with the same keys and values can result in different orders of iteration. This behavior is a surprising byproduct of the way the dictionary's fast hash table is implemented.

[Click here to view code image](#)

```
a = {}
a['foo'] = 1
a['bar'] = 2

# Randomly populate 'b' to cause hash conflicts
while True:
    z = randint(99, 1013)
    b = {}
    for i in range(z):
        b[i] = i
    b['foo'] = 1
    b['bar'] = 2
    for i in range(z):
        del b[i]
    if str(b) != str(a):
        break
```

```
print(a)
print(b)
print('Equal?', a == b)
```

```
>>>
{'foo': 1, 'bar': 2}
{'bar': 2, 'foo': 1}
Equal? True
```

The `OrderedDict` class from the `collections` module is a special type of dictionary that keeps track of the order in which its keys were inserted. Iterating the keys of an `OrderedDict` has predictable behavior. This can vastly simplify testing and debugging by making all code deterministic.

[Click here to view code image](#)

```
a = OrderedDict()
a['foo'] = 1
a['bar'] = 2
b = OrderedDict()
b['foo'] = 'red'
b['bar'] = 'blue'

for value1, value2 in zip(a.values(), b.values()):
    print(value1, value2)

>>>
1 red
2 blue
```

Default Dictionary

Dictionaries are useful for bookkeeping and tracking statistics. One problem with dictionaries is that you can't assume any keys are already present. That makes it clumsy to do simple things like increment a counter stored in a dictionary.

```
stats = {}
key = 'my_counter'
if key not in stats:
    stats[key] = 0
stats[key] += 1
```

The `defaultdict` class from the `collections` module simplifies this by automatically storing a default value when a key doesn't exist. All you have to do is provide a function that will return the default value each time a key is missing. In this example, the `int` built-in function returns `0` (see [Item 23: “Accept Functions for Simple Interfaces Instead of Classes”](#) for another example). Now, incrementing a counter is simple.

```
stats = defaultdict(int)
stats['my_counter'] += 1
```


Heap Queue

Heaps are useful data structures for maintaining a priority queue. The `heapq` module provides functions for creating heaps in standard `list` types with functions like `heappush`, `heappop`, and `nsmallest`.

Items of any priority can be inserted into the heap in any order.

```
a = []
heappush(a, 5)
heappush(a, 3)
heappush(a, 7)
heappush(a, 4)
```

Items are always removed by highest priority (lowest number) first.

[Click here to view code image](#)

```
print(heappop(a), heappop(a), heappop(a), heappop(a))
>>>
3 4 5 7
```

The resulting `list` is easy to use outside of `heapq`. Accessing the `0` index of the heap will always return the smallest item.

[Click here to view code image](#)

```
a = []
heappush(a, 5)
heappush(a, 3)
heappush(a, 7)
heappush(a, 4)
assert a[0] == nsmallest(1, a)[0] == 3
```

Calling the `sort` method on the `list` maintains the heap invariant.

```
print('Before:', a)
a.sort()
print('After: ', a)

>>>
Before: [3, 4, 7, 5]
After:  [3, 4, 5, 7]
```

Each of these `heapq` operations takes logarithmic time in proportion to the length of the list. Doing the same work with a standard Python list would scale linearly.

Bisection

Searching for an item in a `list` takes linear time proportional to its length when you call the `index` method.

```
x = list(range(10**6))
i = x.index(991234)
```

The `bisect` module's functions, such as `bisect_left`, provide an efficient binary search through a sequence of sorted items. The index it returns is the insertion point of the value into the sequence.

```
i = bisect_left(x, 991234)
```

The complexity of a binary search is logarithmic. That means using `bisect` to search a list of 1 million items takes roughly the same amount of time as using `index` to linearly search a list of 14 items. It's way faster!

Iterator Tools

The `itertools` built-in module contains a large number of functions that are useful for organizing and interacting with iterators (see [Item 16](#): “[Consider Generators Instead of Returning Lists](#)” and [Item 17](#): “[Be Defensive When Iterating Over Arguments](#)” for background). Not all of these are available in Python 2, but they can easily be built using simple recipes documented in the module. See `help(itertools)` in an interactive Python session for more details.

The `itertools` functions fall into three main categories:

- Linking iterators together
 - `chain`: Combines multiple iterators into a single sequential iterator.
 - `cycle`: Repeats an iterator's items forever.
 - `tee`: Splits a single iterator into multiple parallel iterators.
 - `zip_longest`: A variant of the `zip` built-in function that works well with iterators of different lengths.
- Filtering items from an iterator
 - `islice`: Slices an iterator by numerical indexes without copying.
 - `takewhile`: Returns items from an iterator while a predicate function returns `True`.
 - `dropwhile`: Returns items from an iterator once the predicate function returns `False` for the first time.
 - `filterfalse`: Returns all items from an iterator where a predicate function returns `False`. The opposite of the `filter` built-in function.
- Combinations of items from iterators
 - `product`: Returns the Cartesian product of items from an iterator, which is a nice alternative to deeply nested list comprehensions.
 - `permutations`: Returns ordered permutations of length `N` with items from an iterator.
 - `combination`: Returns the unordered combinations of length `N` with unrepeated items from an iterator.

There are even more functions and recipes available in the `itertools` module that I don't mention here. Whenever you find yourself dealing with some tricky iteration code, it's worth looking at the `itertools` documentation again to see whether there's anything there for you to use.

Things to Remember

- ◆ Use Python's built-in modules for algorithms and data structures.
- ◆ Don't reimplement this functionality yourself. It's hard to get right.

Item 47: Use `decimal` When Precision Is Paramount

Python is an excellent language for writing code that interacts with numerical data. Python's integer type can represent values of any practical size. Its double-precision floating point type complies with the IEEE 754 standard. The language also provides a standard complex number type for imaginary values. However, these aren't enough for every situation.

For example, say you want to compute the amount to charge a customer for an international phone call. You know the time in minutes and seconds that the customer was on the phone (say, 3 minutes 42 seconds). You also have a set rate for the cost of calling Antarctica from the United States (\$1.45/minute). What should the charge be?

With floating point math, the computed charge seems reasonable.

```
rate = 1.45
seconds = 3*60 + 42
cost = rate * seconds / 60
print(cost)
```

```
>>>
5.364999999999999
```

But rounding it to the nearest whole cent rounds down when you want it to round up to properly cover all costs incurred by the customer.

```
print(round(cost, 2))
```

```
>>>
5.36
```

Say you also want to support very short phone calls between places that are much cheaper to connect. Here, I compute the charge for a phone call that was 5 seconds long with a rate of \$0.05/minute:

```
rate = 0.05
seconds = 5
cost = rate * seconds / 60
print(cost)
```

```
>>>
0.004166666666666667
```

The resulting float is so low that it rounds down to zero. This won't do!

```
print(round(cost, 2))
>>>
0.0
```

The solution is to use the `Decimal` class from the `decimal` built-in module. The `Decimal` class provides fixed point math of 28 decimal points by default. It can go even

higher if required. This works around the precision issues in IEEE 754 floating point numbers. The class also gives you more control over rounding behaviors.

For example, redoing the Antarctica calculation with `Decimal` results in an exact charge instead of an approximation.

[Click here to view code image](#)

```
rate = Decimal('1.45')
seconds = Decimal('222') # 3*60 + 42
cost = rate * seconds / Decimal('60')
print(cost)

>>>
5.365
```

The `Decimal` class has a built-in function for rounding to exactly the decimal place you need with the rounding behavior you want.

[Click here to view code image](#)

```
rounded = cost.quantize(Decimal('0.01'), rounding=ROUND_UP)
print(rounded)

>>>
5.37
```

Using the `quantize` method this way also properly handles the small usage case for short, cheap phone calls. Here, you can see the `Decimal` cost is still less than 1 cent for the call:

[Click here to view code image](#)

```
rate = Decimal('0.05')
seconds = Decimal('5')
cost = rate * seconds / Decimal('60')
print(cost)

>>>
0.00416666666666666666666666666667
```

But the `quantize` behavior ensures that this is rounded up to one whole cent.

[Click here to view code image](#)

```
rounded = cost.quantize(Decimal('0.01'), rounding=ROUND_UP)
print(rounded)

>>>
0.01
```

While `Decimal` works great for fixed point numbers, it still has limitations in its precision (e.g., $1/3$ will be an approximation). For representing rational numbers with no limit to precision, consider using the `Fraction` class from the `fractions` built-in module.

Things to Remember

- ◆ Python has built-in types and classes in modules that can represent practically every type of numerical value.

- ◆ The `Decimal` class is ideal for situations that require high precision and exact rounding behavior, such as computations of monetary values.

Item 48: Know Where to Find Community-Built Modules

Python has a central repository of modules (<https://pypi.python.org>) for you to install and use in your programs. These modules are built and maintained by people like you: the Python community. When you find yourself facing an unfamiliar challenge, the Python Package Index (PyPI) is a great place to look for code that will get you closer to your goal.

To use the Package Index, you'll need to use a command-line tool named `pip`. `pip` is installed by default in Python 3.4 and above (it's also accessible with `python -m pip`). For earlier versions, you can find instructions for installing `pip` on the Python Packaging website (<https://packaging.python.org>).

Once installed, using `pip` to install a new module is simple. For example, here I install the `pytz` module that I used in another item in this chapter (see [Item 45: “Use `datetime` Instead of `time` for Local Clocks](#)”):

[Click here to view code image](#)

```
$ pip3 install pytz
Downloading/unpacking pytz
  Downloading pytz-2014.4.tar.bz2 (159kB): 159kB downloaded
  Running setup.py (...) egg_info for package pytz

Installing collected packages: pytz
  Running setup.py install for pytz

Successfully installed pytz
Cleaning up...
```

In the example above, I used the `pip3` command-line to install the Python 3 version of the package. The `pip` command-line (without the 3) is also available for installing packages for Python 2. The majority of popular packages are now available for either version of Python (see [Item 1: “Know Which Version of Python You’re Using](#)”). `pip` can also be used with `pyenv` to track sets of packages to install for your projects (see [Item 53: “Use Virtual Environments for Isolated and Reproducible Dependencies](#)”).

Each module in the PyPI has its own software license. Most of the packages, especially the popular ones, have free or open source licenses (see <http://opensource.org> for details). In most cases, these licenses allow you to include a copy of the module with your program (when in doubt, talk to a lawyer).

Things to Remember

- ◆ The Python Package Index (PyPI) contains a wealth of common packages that are built and maintained by the Python community.
- ◆ `pip` is the command-line tool to use for installing packages from PyPI.
- ◆ `pip` is installed by default in Python 3.4 and above; you must install it yourself for older versions.

- ◆ The majority of PyPI modules are free and open source software.